Πετράκης Ραφαήλ Ιωάννης ⋆ csd5748

ΗΥ-380: $5^{η}$ Σειρά Ασκήσεων

[**17.7**]

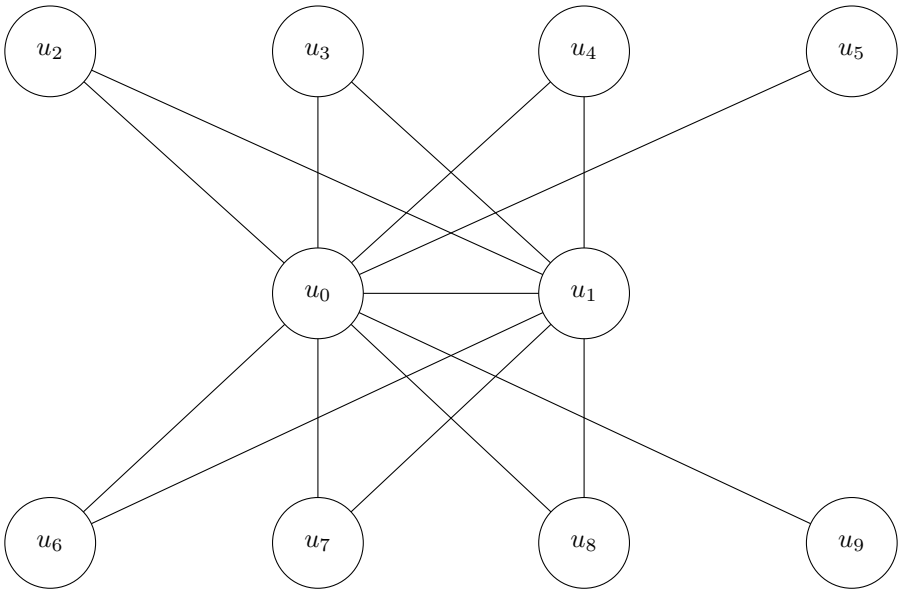
⋆ CLIQUE Problem: έχοντας έναν γράφο $G = (V, E)$ και ενα ακέραιο k, υπάρχει ένα υποσύνολο κόμβων $C \subseteq V$ μεγέθους k στο οποίο κάθε ζευγάρι διαφορετικών κόνβων στο C συνδέονται με μια ακμή του E;
⋆ Certificate: αν η απάντηση είναι «ναι», το πιστοποιητικό είναι το ζητούμενο υποσύνολο κομβων του C που δημιουργεί μια κλίκα.

⋆ Verification Algorithm:
1. Έλεγχος μεγέθους: μετραμε τον αριθμό των κορυφών στο προτεινόμενο υποσύνολο C και επαληθεύουμε ότι $|C| = k$.
- Χρονική πολυπλοκότητα: η μέτρηση των κορυφών απαιτεί χρόνο $O(|C|)$, ο οποίος φράσσεται από $O(|V|)$.
2. Έλεγχος πληρότητας: επαναλαβάνουμε για κάθε δυνατό ζεύγος διαφορετικών κορυφών $u, v \in C$ και ελέγχουμε αν η ακμή (u, v) υπάρχει στο σύνολο ακμών E του γράφου.
- Χρονική πολυπλοκότητα: υπάρχουν $k(k-1)/2$ μοναδικά ζεύγη κορυφών στο C, που είναι το πολύ $O(|V|^2)$ ζεύγη, άρα ο ελεγχος ύπαρξης μιας ακμής απαιτεί το πολύ $O(|V|)$ χρόνο. Επομένως, ο έλεγχος όλων των ζευγών απαιτεί, στη χειρότερη περίπτωση $O(|V|^3)$ χρόνο.

⋆ Συμπέρασμα: ο συνολικός χρόνος που απαιτείται για την επαλήθευση του πιστοποιητικού είναι $O(|V| + |V|^3)$, που απλοποιείται σε $O(|V|^3)$. Τελικα, αφού μπορούμε να επαληθεύσουμε ότι ένα δοσμένο σύνολο κορυφών είναι πράγματι μια k-κλίκα σε πολυωνυμικό χρόνο ως προς το μέγεθος του γράφου εισόδου, το πρόβλημα CLIQUE ανήκει στην κλάση NP.

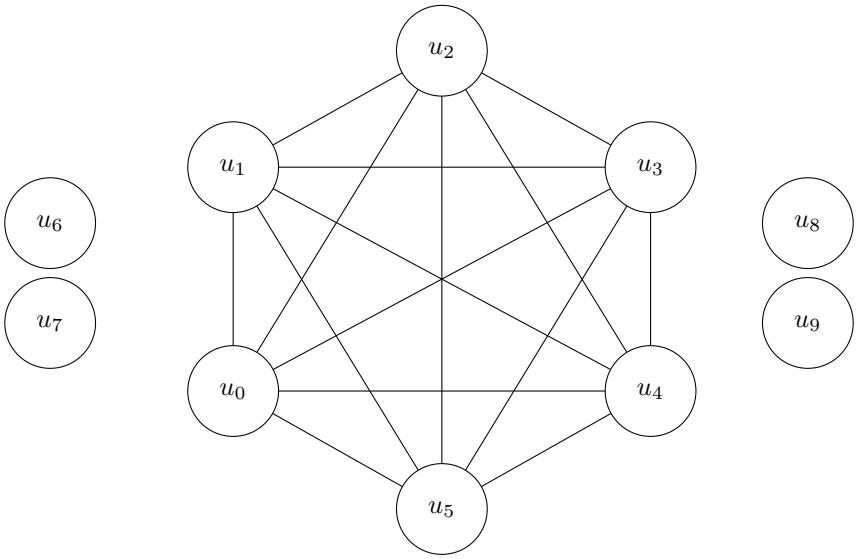
[**17.10**]

⋆ Παράδειγμα γράφου με 10 κορυφές και 15 ακμές, που έχει κάλυψη κορυφών μεγέθους 2 ($\{u_0, u_1\}$):



[**17.11**]

⋆ Παράδειγμα γράφου με 10 κορυφές και 15 ακμές, που έχει μια κλίκα μεγέθους 6 ($\{u_0, \ldots, u_5\}$):



[**17.4**]

⋆ Αποδειξη ότι κάθε γλώσσα L στο P ανάγεται σε πολυωνυμικό χρόνο στη γλώσσα $M = \{5\}$, κατασκευάζοντας μια συνάρτηση αναγωγής $f(x)$ η οποία να υπολογίζεται σε πολυωνυμικό χρόνο.

∗ Αφού $L \in P$, γνωρίζουμε ότι υπάρχει μια ντετερμινιστική μηχανή turing που μπορεί να αποφασίσει αν μια είσοδος x ανήκει στην L σε πολυωνυμικό χρονο. Η συνάρτηση αναγωγής $f(x)$ θα λειτουργεί ως εξής:
1. Προσομοίωση: τρέχει τη μηχανή απόφασης της L για την είσοδο x.
2. Αποδοχή: αν $x \in L$, δηλαδη η μηχανή αποδέχεται, η $f(x)$ επιστρέφει το 5. Έτσι, $f(x) \in M$.
3. Απόρριψη: αν $x \notin L$, δηλαδή η μηχανή απορρίπτει, η $f(x)$ δεν επιστρέφει το 5. Έτσι, $f(x) \notin M$.
∗ Πολυπλοκότητα χρόνου: η προσομοίωση της μηχανής απόφασης απαιτεί πολυωνυμικό χρόνο, και η παραγωγή μιας σταθερής εξόδου απαιτεί σταθερό χρόνο $O(1)$. Άρα, ολόκληρη η συνάρτηση $f(x)$ υπολογίζεται σε πολυωνυμικό χρόνο. Αφού ισχυει $x \in L \iff f(x) \in M$ και η συνάρτηση υπολογίζεται σε πολυωνυμικό χρόνο, αποδεικνύεται ότι $L \leq_p M$.

[**18.2**]

[×] Δεν μπόρεσα να λύσω την συγκεκριμένη άσκηση ...

[**18.11**]

α) Βέλτιστη λύση: είναι το συνολο $\{S_3, S_4, S_5\}$, αφού το στοιχείο 12 ανήκει μόνο στο S_5, άρα είναι υποχρεωτικό, και τα υπόλοιπα 8 στοιχεία που μένουν ακάλυπτα, καλύπτονται πλήρως και με τον μικρότερο δυνατό αριθμό συνόλων επιλέγοντας τα S_4 και S_3.

β) Λύση απληστου αλγορίθμου: ο άπληστος αλγόριθμος παράγει είτη τη λύση $\{S_1, S_4, S_5, S_3\}$ είτε την $\{S_1, S_4, S_5, S_6\}$, ανάλογα με το πώς επιλύεται η ισοβαθμία. Αυτό συμβαίνει επειδή αρχικά επιλέγει το S_1 που καλύπτει 6 στοιχεία, επειτα το S_4 που καλύπτει 3 νέα στοιχεία, μετά το S_5 που καλύπτει 2 νέα στοιχεία, και τέλος, για το μοναδικό στοιχείο που απομένει, επιλέγει το S_3 ή το S_6.

[**18.12**]

#0: Αρχικά, γνωρίζουμε πως σε οποιοδήποτε πεπερασμένο μη κατευθυνόμενο γραφο $G = (V, E)$, κάθε ακμή συνδέει ακριβώς δύο κορυφές. Επομένως, ισχύει: $\sum_{v \in V} deg(v) = 2|E|$. Επειδή το $2|E|$ είναι πολλαπλάσιο του 2, το άθροισμα των βαθμών όλων των κορυφών στο γράφημα πρέπει να είναι ένας άρτιος αριθμός: $\sum_{v \in V} deg(v) = \text{Άρτιος}$.
#1: Μπορούμε να χωρίσουμε το σύνολο όλων των κορυφών σε δύο ξένα μεταξύ τους υποσύνολα: $V_{\text{άρτιες}}$ και $V_{\text{περιττές}}$. Έτσι, έχουμε: $\sum_{v \in V} deg(v) = \sum_{v \in V_{\text{άρτιες}}} deg(v) + \sum_{v \in V_{\text{περιττές}}} deg(v) \implies \text{Άρτιος} = \sum_{v \in V_{\text{άρτιες}}} deg(v) + \sum_{v \in V_{\text{περιττές}}} deg(v)$.
#2: Επιπλέον, το άθροισμα των βαθμών των κορυφών στο $V_{\text{άρτιες}}$ προκύπτει προσθέτοντας άρτιους αριθμούς, αρα θα είναι πάντα άρτιο: $\text{Άρτιος} = \text{Άρτιος} + \sum_{v \in V_{\text{περιττές}}} deg(v)$. Και αφού ισχύει $\text{Άρτιος} - \text{Άρτιος} = \text{Άρτιος}$, ειναι: $\sum_{v \in V_{\text{περιττές}}} deg(v) = \text{Άρτιος}$.
#3: Συμπερασματικά, κάθε όρος στο άθροισμα $\sum_{v \in V_{\text{περιττές}}} deg(v)$ είναι ένας περιττός αριθμός.Το άθροισμα ενός συνόλου περιττών αριθμών δΙνει άρτιο αποτέλεσμα μόνο εάν προστίθεται ένα άρτιο πλήθος όρων. Επομένως, το πλήθος των στοιχείων του συνόλου $V_{\text{περιττές}}$ (δηλαδή το πλήθος των κορυφών με περιττό βαθμό) πρέπει να είναι υποχρεωτικά άρτιο ...

[**18.2**]

⋆ Ψευδοκώδικας:

// Έστω $G = (V, E)$ ένας γραφος που αναπαριστάται μέσω ενός πίνακα γειτνίασης, και N ο συνολικός αριθμός των κορυφών.

Αλγόριθμος FindHamiltonianCycle (G, N):

```
path = πίνακας μεγέθους N γεμάτος με -1 // Αρχικοποίηση.
path[0] = 0 // Ξεκινάμε τον κύκλο τυχαια από την κορυφή 0.
// Ξεκινάμε την αναδρομική αναζήτηση από τη 2η θέση του μονοπατιού:
Αν Backtrack(G, path, 1, N) είναι αληθές: επέστρεψε path
Αλλιώς: επέστρεψε «Δεν υπάρχει Κύκλος Hamilton ...»

Συνάρτηση Backtrack(G, path, pos, N):
    // Βασική συνθήκη τερματισμού αν έχουν τοποθετηθεί όλες οι κορυφές στο μονοπάτι:
    Αν pos == N:
        // Έλεγχος αν η τελευταία κορυφή συνδέεται πίσω με την αρχική:
        Αν G[path[N-1]][path[0]] == 1: επέστρεψε «αληθές»
        Αλλιώς: επέστρεψε «ψευδές»
    // Δοκιμάζουμε όλες τις υπόλοιπες κορυφές ως υποψήφιες:
    Για v από 1 έως N-1:
        Αν IsValid(v, G, path, pos):
            path[pos] = v // Προσθήκη της κορυφής στο μονοπάτι.
            Αν Backtrack(G, path, pos + 1, N) αληθές: επέστρεψε «αληθές» // Αναδρομική κλήση για το υπολοιπο.
            path[pos] = -1 // Οπισθοδρόμηση: αφαιρούμε την κορυφή αν οδηγήσει σε αδιέξοδο.
    Επέστρεψε «ψευδές»

Συνάρτηση IsValid(v, G, path, pos):
    // Συνθήκη 1: ελέγχουμε αν υπάρχει ακμή από την προηγούμενη κορυφή προς την v:
    Αν G[path[pos-1]][v] == 0: επέστρεψε «ψευδές»
    // Συνθήκη 2: ελέγχουμε αν η κορυφή v υπάρχει ήδη στο τρέχον μονοπάτι:
    Για i από 0 έως pos-1:
        Αν path[i] == v: επέστρεψε «ψευδές»
    Επέστρεψε «αληθές»

Τέλος Αλγοριθμου.
```

ΤΕΛΟΣ

LaTeX